# Chapter 3

# Making Decisions

**By**

**Dr. Sara Tarik Kamal**

# Decision Structures and the if Statement

- In C#, you use the if statement to write a single-alternative decision structure. Here is the general format of the if statement:

```
if (expression)
{
   statement;
   statement;
   etc.
}
```

- The expression that appears inside the parentheses is a Boolean expression.
- if statement that has only one conditionally executed statement, can be written in the following general format:

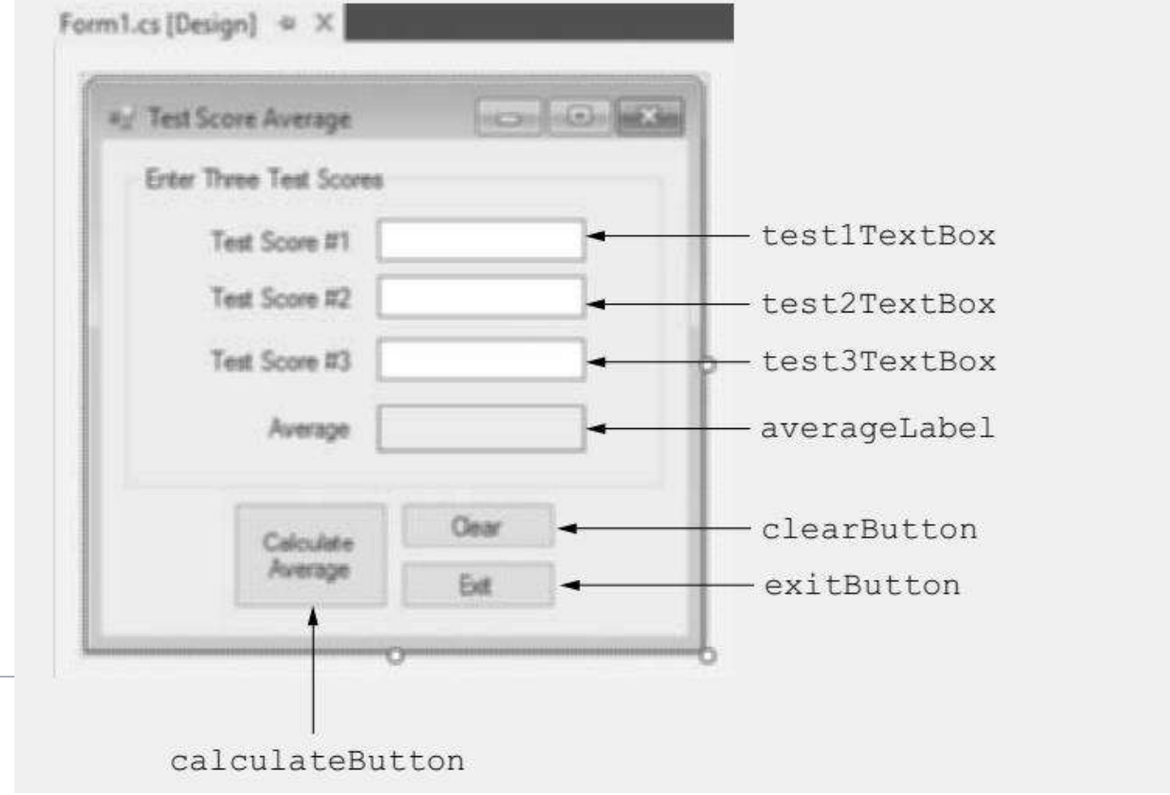
```
if (expression)
   statement;
```

# Example 1

## *Test Score Average application*

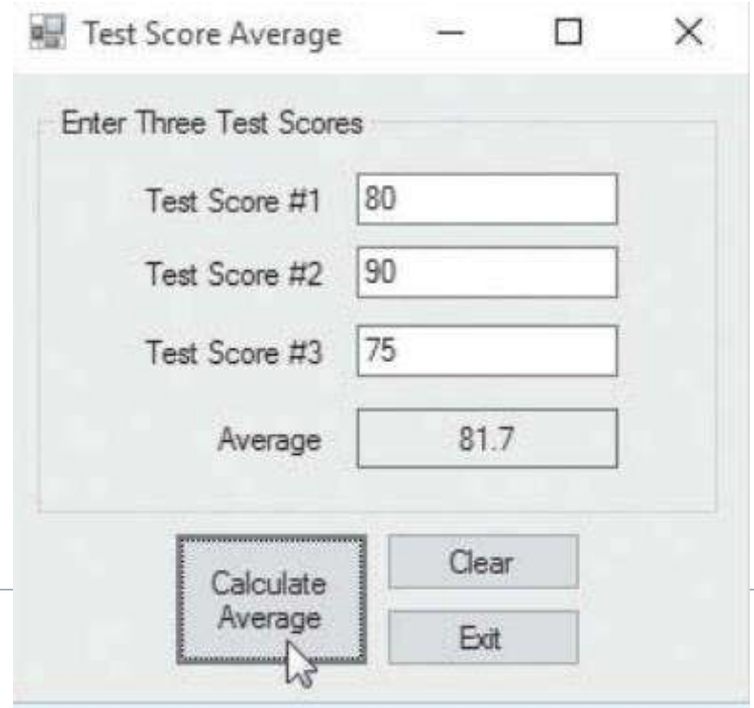
**Allows the user to enter three test scores and calculates the average of the test scores.**

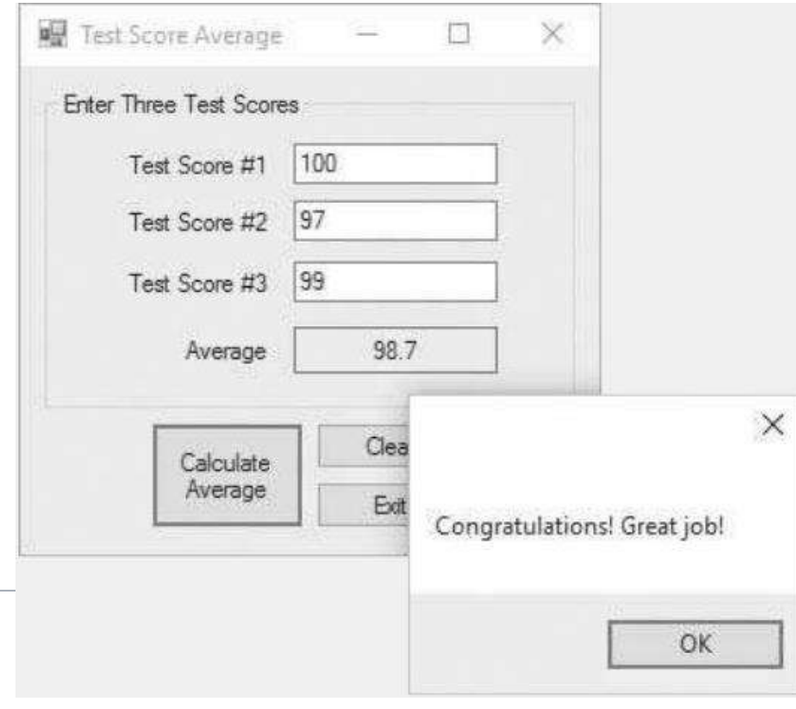
**If the average is greater than 95, the application also displays a message congratulating to the user.**



# Example 1

- First, enter the following test scores in the TextBoxes: 80, 90, and 75. Click the Calculate Average button and the average should appear as shown in the left Figure.
- Next, click the Clear button to clear the TextBoxes and the average. Now, enter the following test scores in the TextBoxes: 100, 97, and 99. Click the Calculate Average button. This time, in addition to displaying the average, the application displays the message box shown in the right Figure.



# Example 1- Form1 code for Test Score Average application

```
private void calculateButton_Click(object sender, EventArgs e)
{
    try
    {
        const double HIGH_SCORE = 95.0;     // High Score value
        double test1, test2, test3, average;  // Variables

        // Get the test scores from the TextBoxes.
        test1 = double.Parse(test1TextBox.Text);
        test2 = double.Parse(test2TextBox.Text);
        test3 = double.Parse(test3TextBox.Text);

        // Calculate the average test score.
        average = (test1 + test2 + test3) / 3.0;

        // Display the average, rounded to 2 decimal places.
        averageLabel.Text = average.ToString("n1");

        // If the average is a high score, congratulate
        // the user with a message box.
        if (average > HIGH_SCORE)
        {
            MessageBox.Show("Congratulations! Great job!");
        }
    }
    catch (Exception ex)
    {
        // Display the default error message.
        MessageBox.Show(ex.Message);
    }
}
```

# Example 1- Form1 code for Test Score Average application

```
private void clearButton_Click(object sender, EventArgs e)
{
    // Clear the TextBoxes and the averageLabel control.
    test1TextBox.Text = "";
    test2TextBox.Text = "";
    test3TextBox.Text = "";
    averageLabel.Text = "";

    // Reset the focus to test1.
    test1TextBox.Focus();
}
```

# if-else Statement

- The general format of the if-else statement in C# is:

```
if (expression)
{
    statement;
    statement;
    etc.
}
else
{
    statement;
    statement;
    etc.
}
```

If the Boolean expression is true, this set of statements is executed.

If the Boolean expression is false, this set of statements is executed.

- The following general format shows only one statement following the if clause and only one statement following the else clause:

```
if (expression)
    statement;
else
    statement;
```

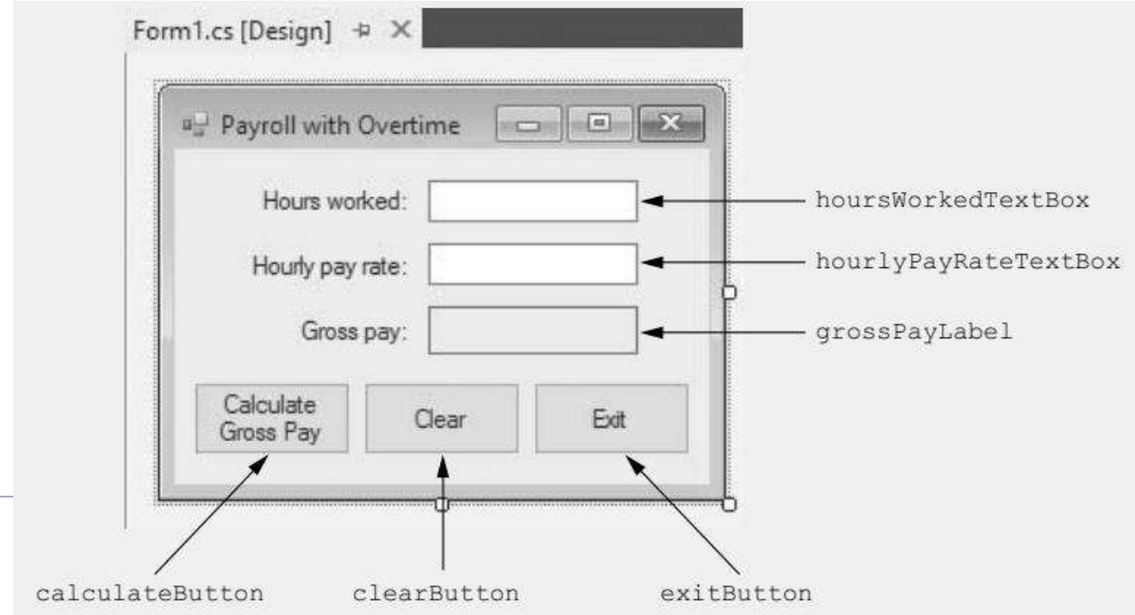
# Example 2

***Payroll with Overtime application***

**Create a payroll application that calculates an employee's gross pay, including overtime pay, if an employee works more than 40 hours in a week, it is said that the employee has worked overtime.**

**For example, an employee that has worked 45 hours in a week has worked 5 overtime hours.**

**Employees that work overtime get paid their regular hourly pay rate for the first 40 hours plus 1.5 times their regular hourly pay rate for all hours over 40.**



# Example 2

- The application allows the user to enter the number of hours worked and the hourly pay rate into TextBoxes.

- When the user clicks a button, the gross pay is calculated in the following manner:

If the hours worked is greater than 40:
    base pay = hourly pay rate × 40
    overtime hours = hours worked – 40
    overtime pay = overtime hours × hourly pay rate × 1.5
    gross pay = base pay + overtime pay
Else:
    gross pay = hours worked × hourly pay rate

# Example 2- Form1 code for Payroll with Overtime application

```
private void calculateButton_Click(object sender, EventArgs e)
{
    try
    {
        // Named constants
        const decimal BASE_HOURS = 40m;
        const decimal OT_MULTIPLIER = 1.5m;

        // Local variables
        decimal hoursWorked;    // Number of hours worked
        decimal hourlyPayRate; // Hourly pay rate
        decimal basePay;        // Pay not including overtime
        decimal overtimeHours;  // overtime hours worked
        decimal overtimePay;    // overtime pay
        decimal grossPay;       // total gross pay

        // Get the hours worked and hourly pay rate.
        hoursWorked = decimal.Parse(hoursWorkedTextBox.Text);
        hourlyPayRate = decimal.Parse(hourlyPayRateTextBox.Text);
```

# Example 2- Form1 code for Payroll with Overtime application

```
        // Determine the gross pay.
        if (hoursWorked > BASE_HOURS)
        {
            // Calculate the base pay (without overtime).
            basePay = hourlyPayRate * BASE_HOURS;

            // Calculate the number of overtime hours.
            overtimeHours = hoursWorked - BASE_HOURS;

            // Calculate the overtime pay.
            overtimePay = overtimeHours * hourlyPayRate *
                OT_MULTIPLIER;

            // Calculate the gross pay.
            grossPay = basePay + overtimePay;
        }
        else
        {
            // Calculate the gross pay.
            grossPay = hoursWorked * hourlyPayRate;
        }

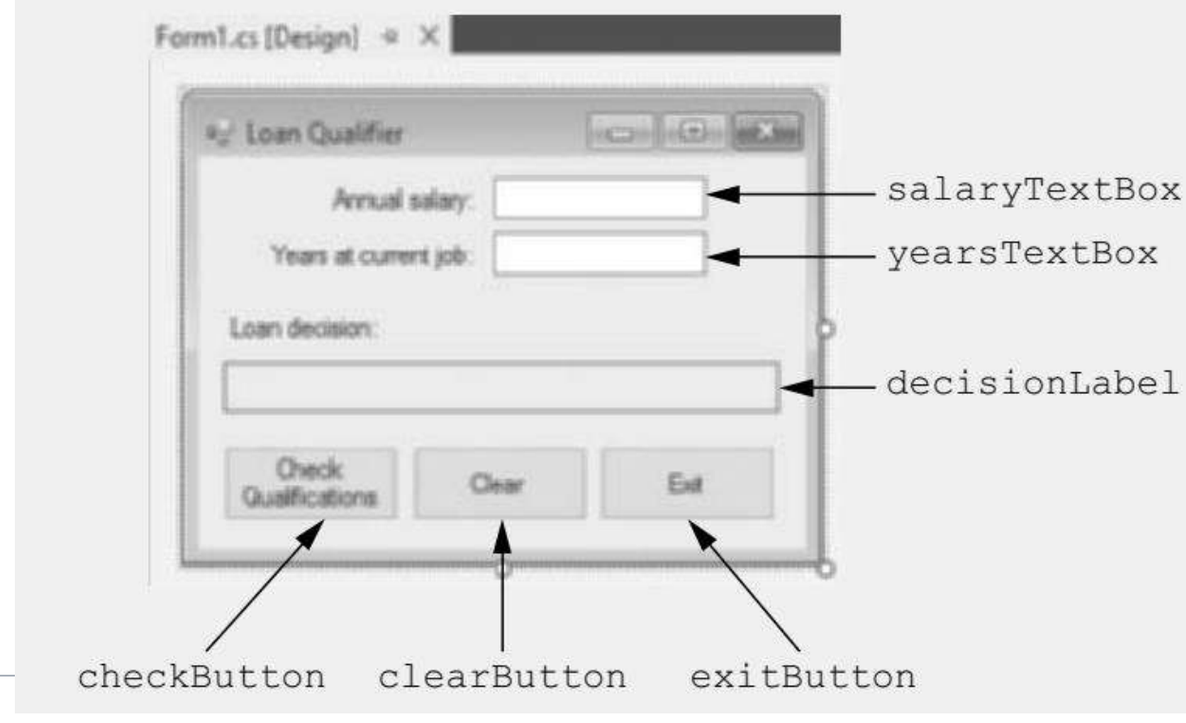
        // Display the gross pay.
        grossPayLabel.Text = grossPay.ToString("c");
    }
    catch (Exception ex)
    {
        // Display an error message.
        MessageBox.Show(ex.Message);
    }
```

# Example 3

## *Loan Qualifier application*

**Determine whether a person qualifies for a loan.**

**To qualify for the loan, the person must earn a salary of at least $40,000 and must have been employed at his or her current job for at least 2 years.**



# Example 3- Form1 code for Loan Qualifier application

```
private void checkButton_Click(object sender, EventArgs e)
{
    try
    {
        // Names constants
        const decimal MINIMUM_SALARY = 40000m;
        const int MINIMUM_YEARS_ON_JOB = 2;

        // Local variables
        decimal salary;
        int yearsOnJob;

        // Get the salary and years on the job.
        salary = decimal.Parse(salaryTextBox.Text);
        yearsOnJob = int.Parse(yearsTextBox.Text);

        // Determine whether the user qualifies.
        if (salary >= MINIMUM_SALARY)
        {
            if (yearsOnJob >= MINIMUM_YEARS_ON_JOB)
            {
                // The user qualifies.
                decisionLabel.Text = "You qualify for the loan.";
            }
            else
            {
                // The user does not qualify.
                decisionLabel.Text = "Minimum years at current " +
                    "job not met.";
            }
        }
        else
        {
            // The user does not qualify.
            decisionLabel.Text = "Minimum salary requirement " +
                "not met.";
        }
    }
    catch (Exception ex)
    {
        // Display an error message.
        MessageBox.Show(ex.Message);
    }
}
```

# Example 3- Form1 code for Loan Qualifier application

```
private void clearButton_Click(object sender, EventArgs e)
{
    // Clear the TextBoxes and the decisionLabel.
    salaryTextBox.Text = "";
    yearsTextBox.Text = "";
    decisionLabel.Text = "";

    // Reset the focus.
    salaryTextBox.Focus();
}

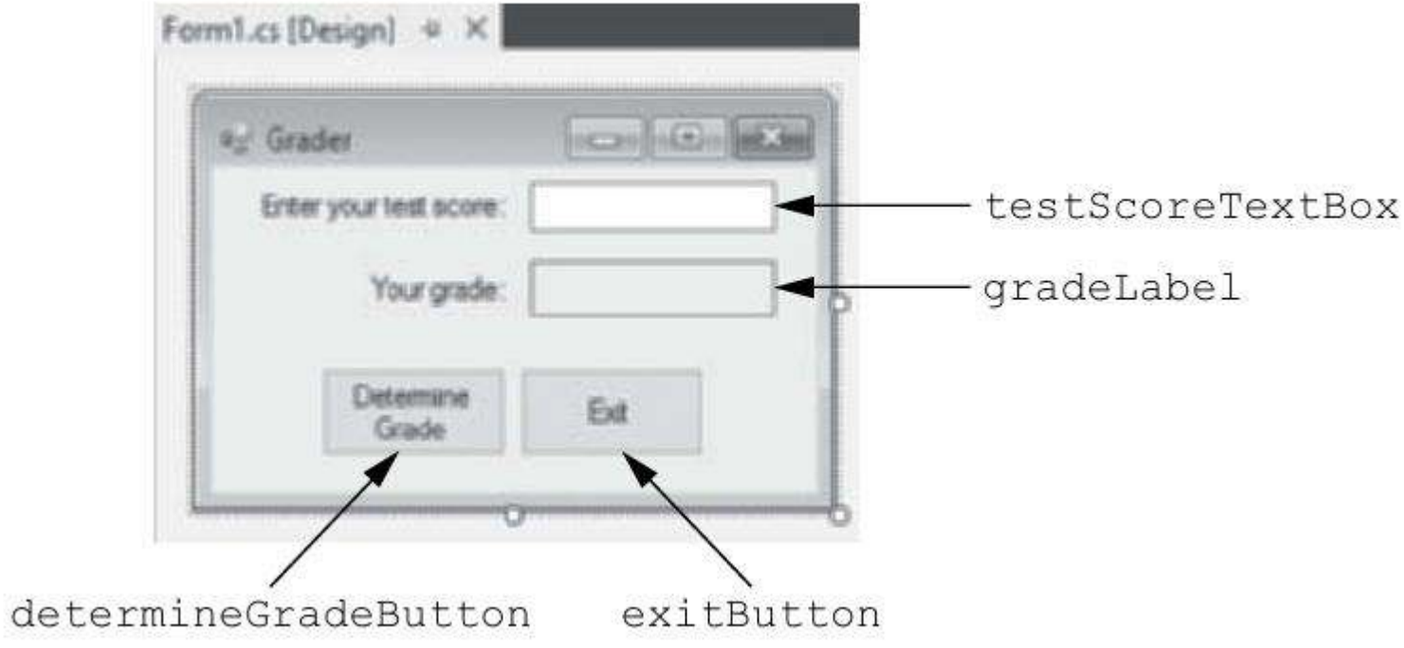
private void exitButton_Click(object sender, EventArgs e)
{
    // Close the form.
    this.Close();
}
```

# Example 4

***Grader application***

- When you run the application, you enter a numeric test score into the testScoreTexBox control and click the determineGradeButton control; a grade is then displayed in the gradeLabel control.
- The following 10-point grading scale is used to determine the grade:

| Test Score | Grade |
|---|---|
| 90 and above | A |
| 80–89 | B |
| 70–79 | C |
| 60–69 | D |
| Below 60 | F |



# Example 4- Form1 code for Grader application

```
private void determineGradeButton_Click(object sender, EventArgs e)
{
    try
    {
        // Variable to hold the test score.
        double testScore;

        // Get the test score.
        testScore = double.Parse(testScoreTextBox.Text);

        // Determine the grade.
        if (testScore < 60)
        {
            gradeLabel.Text = "F";
        }
        else
        {
            if (testScore < 70)
            {
                gradeLabel.Text = "D";
            }
            else
            {
                if (testScore < 80)
                {
                    gradeLabel.Text = "C";
                }
                else
                {
                    if (testScore < 90)
                    {
                        gradeLabel.Text = "B";
                    }
                    else
                    {
                        gradeLabel.Text = "A";
                    }
                }
            }
        }
    }
    catch (Exception ex)
    {
        // Display an error message.
        MessageBox.Show(ex.Message);
    }
}
```

# if-else-if Statement

- Even though the Grader application previously shown is a simple example, the logic of the nested decision structure is fairly complex.
- C# provides a special version of the decision structure known as the if-else-if statement, which makes this type of logic simpler to write.

```
if (BooleanExpression_1)
{
    statement;
    statement;
    etc.
}
else if (BooleanExpression_2)
{
    statement;
    statement;
    etc.
}

Insert as many else if clauses as necessary...

else
{
    statement;
    statement;
    etc.
}
```

If *BooleanExpression_1* is true, this set of statements is executed.

If *BooleanExpression_2* is true, this set of statements is executed.

This set of statements is executed if none of the Boolean expressions are true.

# Grader application- if-else-if Statement

```
1  private void determineGradeButton_Click(object sender, EventArgs e)
2  {
3      try
4      {
5          // Variable to hold the test score.
6          double testScore;
7
8          // Get the test score.
9          testScore = double.Parse(testScoreTextBox.Text);
10
11         // Determine the grade.
12         if (testScore < 60)
13         {
14             gradeLabel.Text = "F";
15         }
16         else if (testScore < 70)
17         {
18             gradeLabel.Text = "D";
19         }
20         else if (testScore < 80)
21         {
22             gradeLabel.Text = "C";
23         }
24         else if (testScore < 90)
25         {
26             gradeLabel.Text = "B";
27         }
28         else
29         {
30             gradeLabel.Text = "A";
31         }
32     }
33     catch (Exception ex)
34     {
35         // Display an error message.
36         MessageBox.Show(ex.Message);
37     }
38 }
```

# Logical Operators

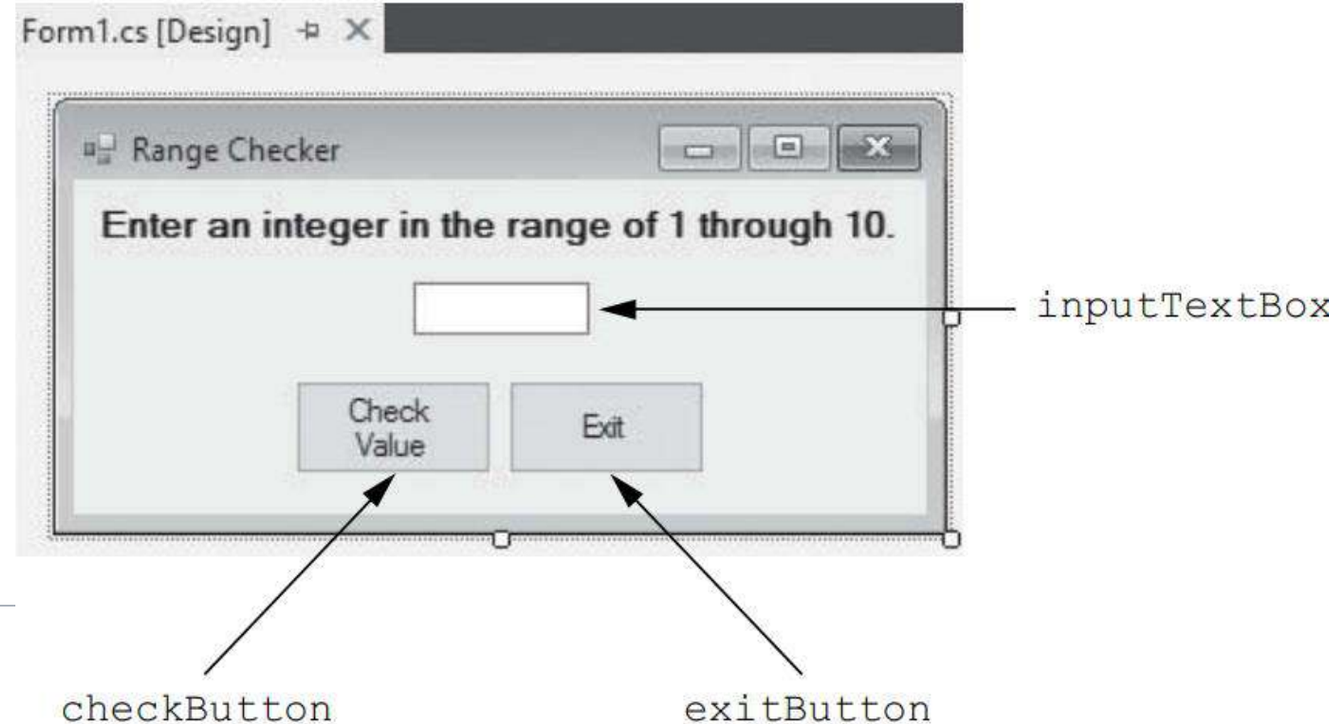
- The C# language provides a set of operators known as logical operators, which you can use to create complex Boolean expressions as shown in the following table

| Operator | Meaning |
|---|---|
| && | This is the logical AND operator. It connects two Boolean expressions into one compound expression. Both subexpressions must be true for the compound expression to be true. |
| \|\| | This is the logical OR operator. It connects two Boolean expressions into one compound expression. One or both subexpressions must be true for the compound expression to be true. It is necessary for only one of the subexpressions to be true, and it does not matter which. |
| ! | This is the logical NOT operator. It is a unary operator, meaning it works with only one operand. The operand must be a Boolean expression. The not operator reverses the truth of its operand. If it is applied to an expression that is true, the operator returns false. If it is applied to an expression that is false, the operator returns true. |

# Example 5

***Range Checker application***

**When you run the application, you enter an integer into the inputTexBox control and click the checkButton control. If you enter a number in the range of 1 through 10, a message box appears letting you know that the number is acceptable. Otherwise, a message box appears letting you know that the number is not acceptable.**



# Example 5- Form1 code for Range Checker application

```
1  private void checkButton_Click(object sender, EventArgs e)
2  {
3      try
4      {
5          // Declare a variable and initialize it with
6          // the user's input.
7          int number = int.Parse(inputTextBox.Text);
8
9          // Check the number's range.
10         if (number >= 1 && number <= 10)
11         {
12             MessageBox.Show("That number is acceptable.");
13         }
14         else
15         {
16             MessageBox.Show("That number is NOT acceptable.");
17         }
18     }
19     catch (Exception ex)
20     {
21         // Display an error message.
22         MessageBox.Show(ex.Message);
23     }
24 }
```

# bool Variables

- The C# language provides the bool data type that you can use to create variables that hold true or false values.
- Example:

```
bool grandMaster;
if (points > 5000)
{
    grandMaster = true;
}
else
{
    grandMaster = false;
}
```

# Flags

- Variables of the bool data type are commonly used as flags.

```
if (grandMaster)
{
    powerLevel += 500;
}
```

- This code performs the following: if grandMaster is true, add 500 to powerLevel.

- Here is another example:

```
if (!grandMaster)
{
   powerLevel = 100;
}
```

This code performs the following: if grandMaster is not true, set powerLevel to 100.

# Comparing Strings

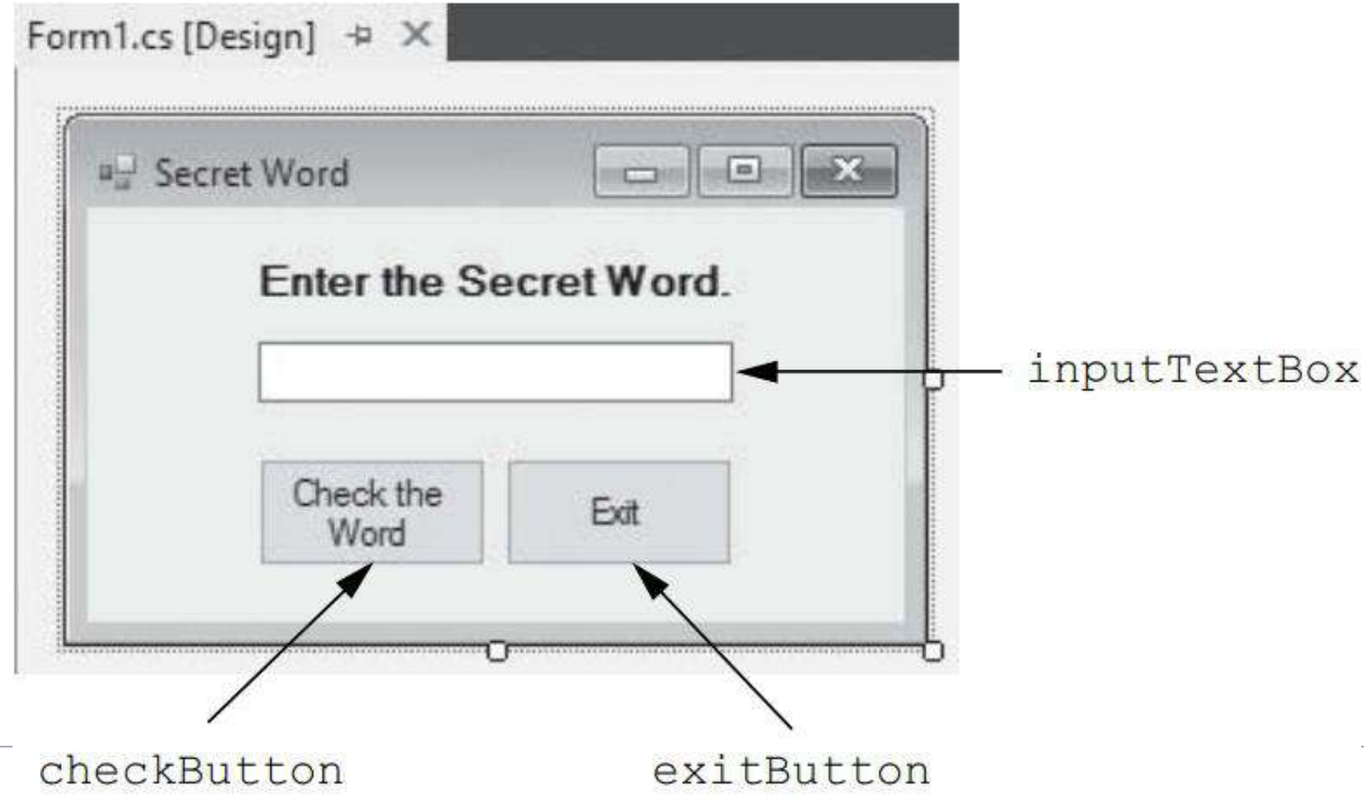
- You can use the == operator to compare two strings.
- Example:

```
string name1 = "Mary";
string name2 = "Mark";
if (name1 == name2)
{
   MessageBox.Show("The names are the same.");
}
else
{
   MessageBox.Show("The names are NOT the same.");
}
```

# Example 6- Form1 code for

## *Secret Word application*

**The form prompts you to enter the secret word into the inputTexBox control. When you click the checkButton control, the application compares the string that you entered to "Ariel."**



# Example 6- Form1 code for Secret Word application

```
1  private void checkButton_Click(object sender, EventArgs e)
2  {
3      // Declare a string variable and initialize it with
4      // the user's input.
5      string secretWord = inputTextBox.Text;
6
7      // Did the user enter the correct secret word?
8      if (secretWord == "Ariel")
9      {
10         MessageBox.Show("That is the correct secret word.");
11     }
12     else
13     {
14         MessageBox.Show("Sorry, that is NOT the secret word.");
15     }
16 }
```

| | |
|---|---|
| `UseSystemPasswordChar` | When `true`, the `TextBox` becomes a password `TextBox`, and the system-specified character masks each character the user types. |

# Case Sensitive Comparison

- The strings “Joe” and “joe” are not equal because the case of the first character is different in each.
- You can pass the Boolean value true as an optional third argument to the String.Compare method if you want it to perform a case insensitive comparison.

  String.Compare(string1, string2)
  - The method returns Zero if string1 is equal to string2.
- Example:

```
string str1 = "JOE";
string str2 = "joe";
if (String.Compare(str1, str2, true) == 0)
{
MessageBox.Show(str1 + " and " + str2 + " are equal.");
}
else
{
MessageBox.Show(str1 + " and " + str2 + " are NOT equal.");
}
```

**Output**

JOE and joe are equal

# Preventing Data Conversion Exceptions with the TryParse Methods

- In the previous chapter, you learned that the Parse methods throw an exception when you try to use them to convert nonnumeric data to a numeric data type.
- To handle the exceptions that are caused by the Parse methods, we have been using the try-catch statement.
- It is a better programming practice to prevent an exception instead of allowing it to happen and then letting a try-catch statement react to it.
- You should use try-catch statements primarily for those exceptions that are beyond your control.

# TryParse Method

- With the TryParse methods, you can determine whether a string (such as a control's Text property) contains a value that can be converted to a specific data type before it is converted to that data type.
- The TryParse methods do not throw an exception, so you can use them without a try-catch statement.
- There are several TryParse methods in the .NET Framework

  We use the int.TryParse method to convert a string to an int.
  We use the double.TryParse method to convert a string to a double.
  We use the decimal.TryParse method to convert a string to a decimal.
- When you call one of the TryParse methods, you pass two arguments:

  1. the string that you want to convert
  2. the name of the variable in which you want to store the converted value.
- The general format of how the int.TryParse method is called:

int.TryParse(string, out targetVariable)

# TryParse Method

- The TryParse methods return either true or false, they are commonly called as the Boolean expression in an if statement.

```
1  int number;
2
3  if (int.TryParse(inputTextBox.Text, out number))
4  {
5      MessageBox.Show("Success!");
6  }
7  else
8  {
9      MessageBox.Show("Enter a valid integer.");
10 }
```

- If inputTextBox.Text is successfully converted to an int, the resulting value is assigned to the number variable, and the method returns true. That causes the statement in line 5 to execute.
- If inputTextBox.Text cannot be converted to an int, the value 0 is assigned to the number variable, and the method returns false. That causes the statement in line 9 (after the else clause) to execute.

# Validating the Data in Multiple TextBoxes

- If a form has multiple TextBoxes, then the user has multiple opportunities to enter an invalid piece of data.
- A well-designed program should validate the contents of each TextBox individually.
- When a piece of invalid data is found, the program should display an error message that tells the user specifically which TextBox contains the bad input.

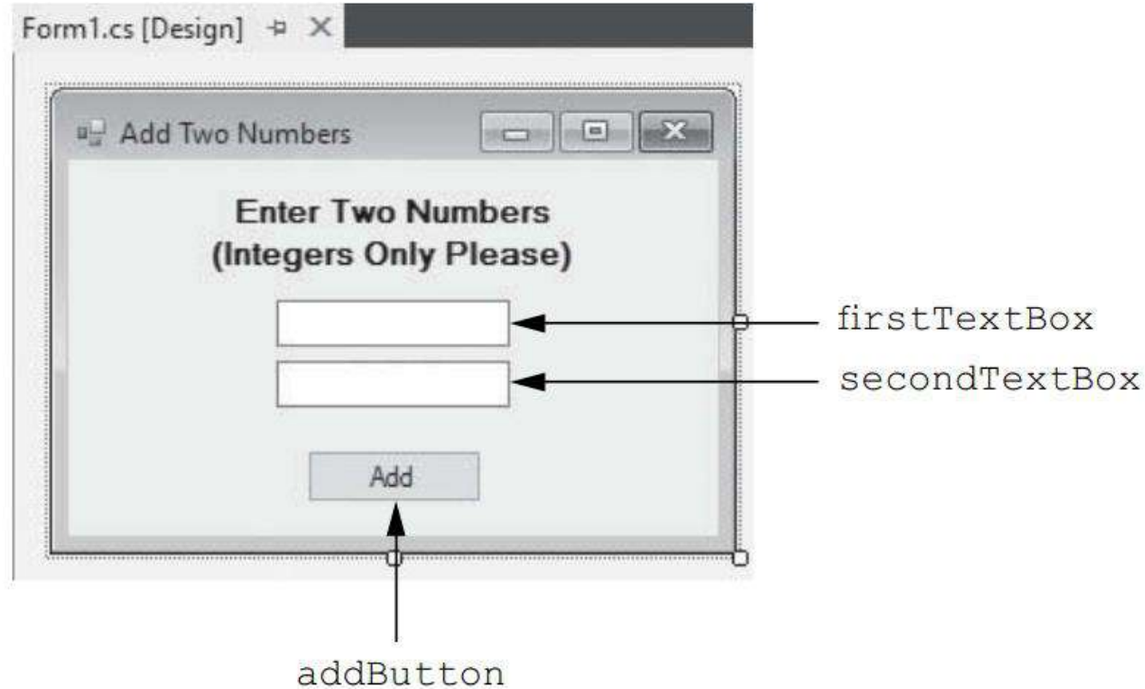
# Example 7

## *Add Two Numbers Application*

**Enter an integer into each of the TextBox controls and then click the Add button.**

**A message box will appear showing the sum of the two numbers.**

**If you enter anything other than an integer into either TextBox, an error message will appear telling you which TextBox contains the invalid data**



# Example 7- Form1 code for Add Two Numbers Application

```
1  private void addButton_Click(object sender, EventArgs e)
2  {
3      // Local variables
4      int first, second, sum;
5
6      if (int.TryParse(firstTextBox.Text, out first))
7      {
8          if (int.TryParse(secondTextBox.Text, out second))
9          {
10             // Add the two numbers and display the sum.
11             sum = first + second;
12             MessageBox.Show(sum.ToString());
13         }
14         else
15         {
16             // Display an error message about the second TextBox.
17             MessageBox.Show("The second TextBox contains invalid data.");
18         }
19     }
20     else
21     {
22         // Display an error message about the first TextBox.
23         MessageBox.Show("The first TextBox contains invalid data.");
24     }
25 }
```

# Input Validation

- Input validation is the process of inspecting data that has been input to a program to make sure it is valid before it is used in a computation.
- The TryParse methods is used to validate the type of data entered by the user.
- You should also validate the accuracy of the data that is entered by the user
- For example,
  - consider a payroll program that accepts the number of hours that an employee has worked in a given week as input.
  - If the payroll clerk accidentally enters 400 hours instead of 40 hours, an unusually large check will be written because there are less than 400 hours in a week! The computer, however, is unaware of this fact, and unless the program is written to catch such errors, it will process the bad data just as if it were good data.

# Input Validation

- For example, in a payroll program we might validate the number of hours worked:

```
 1 if (int.TryParse(hoursWorkedTextBox.Text, out hours))
 2 {
 3     if (hours > 0 && hours <= 168)
 4     {
 5         // Continue to process the input.
 6     }
 7     else
 8     {
 9         MessageBox.Show("Invalid number of hours entered.");
10     }
11 }
12 else
13 {
14     MessageBox.Show("The hours worked must be an integer.");
15 }
```

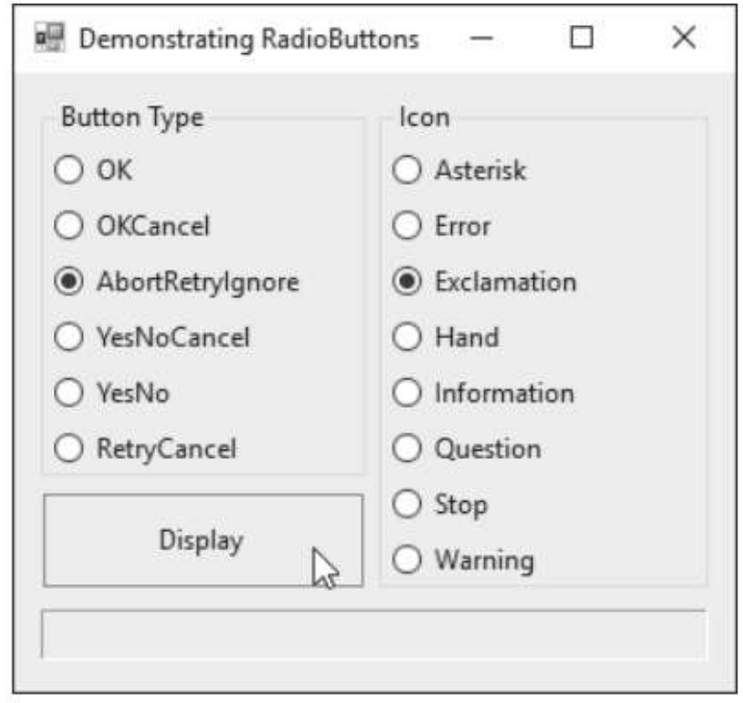
# Input Validation (Example)

- The following code comes from an application that gets a test score as input.
- A valid test score is an integer in the range of 0 through 100.

```
 1 if (int.TryParse(testScoreTextBox.Text, out testScore))
 2 {
 3     if (testScore >= 0 && testScore <= 100)
 4     {
 5         // Continue to process the input.
 6     }
 7     else
 8     {
 9         MessageBox.Show("Test score must be in the range 0   100.");
10     }
11 }
12 else
13 {
14     MessageBox.Show("The test score must be an integer.");
15 }
```

# Radio Buttons

- Radio buttons are useful when you want the user to select one choice from several possible choices.
- Only one radio button in a group may be selected at a time.



# Radio Buttons

- When you want to create a group of radio buttons on a form, you use the RadioButton control, which is found in the Common Controls section of the Toolbox .
- RadioButton controls are normally grouped in one of the following ways:
  - You place them inside a GroupBox control. All RadioButton controls that are inside a GroupBox are members of the same group.
  - You place them inside a Panel control. All RadioButton controls that are inside a Panel are members of the same group.
  - You place them on a form but not inside a GroupBox or a Panel. All RadioButton controls that are on a form but not inside a GroupBox or Panel are members of the same group.

# Radio Buttons

- The RadioButton Control's Text Property: holds the text that is displayed next to the radio button's circle.
- The RadioButton Control's Checked Property:
  - Checked property that determines whether the control is selected or deselected.
  - The Checked property is a Boolean property, which means that it may be set to either True or False.
  - When the Checked property is set to True, the RadioButton is selected, and when the Checked property is set to False, the RadioButton is deselected.
  - By default, the Checked property is set to False.

# Working with Radio Buttons in Code

- You can determine whether a RadioButton control is selected by testing its Checked property.
- For example, suppose a form has a RadioButton control named choice1RadioButton .
- The following if statement determines whether it is selected:

```
if (choice1RadioButton.Checked)
{
MessageBox.Show("You selected Choice 1");
}
```

- Notice that we did not have to use the == operator to explicitly compare the Checked property to the value true .
- This code is equivalent to the following:

```
if (choice1RadioButton.Checked == true)
{
MessageBox.Show("You selected Choice 1.");
}
```

# RadioButton (Example)

- When you run the application, select one of the radio buttons and then click the OK button. A message box appears showing the sport that you selected.

```
1 private void okButton_Click(object sender, EventArgs e)
2 {
3     if (footballRadioButton.Checked)
4     {
5         MessageBox.Show("You selected Football.");
6     }
7     else if (basketballRadioButton.Checked)
8     {
9         MessageBox.Show("You selected Basketball.");
10    }
11    else if (baseballRadioButton.Checked)
12    {
13        MessageBox.Show("You selected Baseball.");
14    }
15 }
```

# Problem

***Color Mixer Application***

The colors red, blue, and yellow are known as the primary colors because they cannot be made by mixing other colors.

When you mix two primary colors, you get a secondary color, as shown here:

- When you mix red and blue, you get purple.
- When you mix red and yellow, you get orange.
- When you mix blue and yellow, you get green.

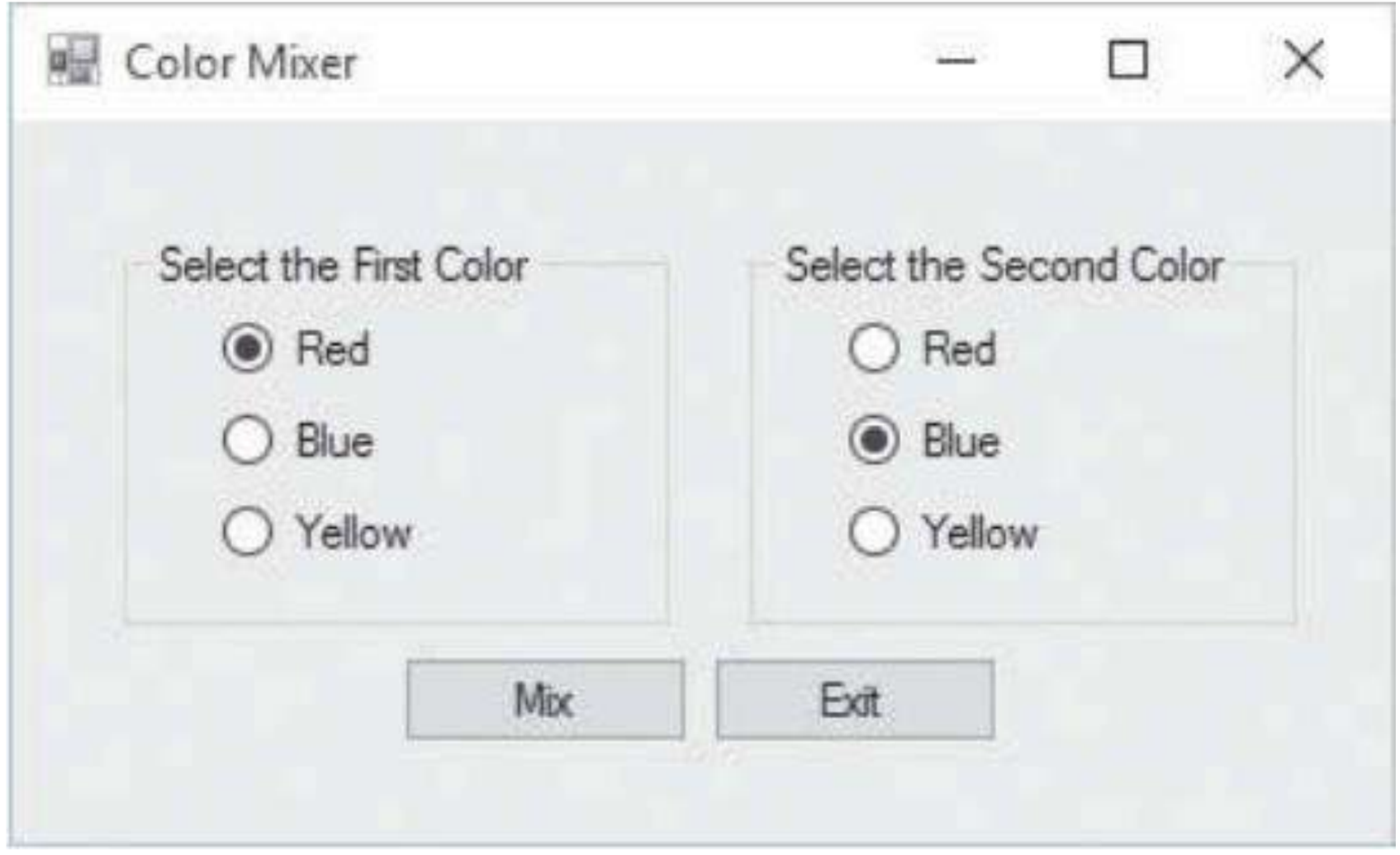
If the user picks the same color from both sets of Radio buttons, set the form's background to that color.

Create an application that lets the user select two primary colors from two different sets of Radio buttons.

The form should also have a Mix button.

When the user clicks the Mix button, the form's background should change to the color that you get when you mix the two selected primary colors.

# Color Mixer form



# CheckedChanged Event

- Any time a RadioButton or a CheckBox control's Checked property changes, a CheckedChanged event happens for that control.
- If you want some action to immediately take place when the user selects (or deselects) a RadioButton or CheckBox control, you can create a CheckedChanged event handler for the control and write the desired code in that event handler.
- To create a CheckedChanged event handler for a RadioButton or a CheckBox, simply double-click the control in the Designer.
- An empty CheckedChanged event handler is created in the code editor

# Example 8

***Color Theme application***

**Create an application that allows the user to select a color using RadioButton controls.**

**When the user selects a color, the form's background color is changed to that color immediately.**

# Example 8- Control property settings

| Control Name | Control Type | Property Settings |
|---|---|---|
| `colorGroupBox` | GroupBox | **Text:** *Select a Background Color* |
| `yellowRadioButton` | RadioButton | **Text:** *Yellow*<br>**Checked:** False |
| `redRadioButton` | RadioButton | **Text:** *Red*<br>**Checked:** False |
| `whiteRadioButton` | RadioButton | **Text:** *White*<br>**Checked:** False |
| `normalRadioButton` | RadioButton | **Text:** *Back to Normal*<br>**Checked:** True |
| `exitButton` | Button | **Text:** *Exit* |

# Example 8- Form1 code for Color Theme application

```
private void yellowRadioButton_CheckedChanged(object sender, EventArgs e)
{
      if (yellowRadioButton.Checked)
      {
          this.BackColor = Color.Yellow;
      }
}

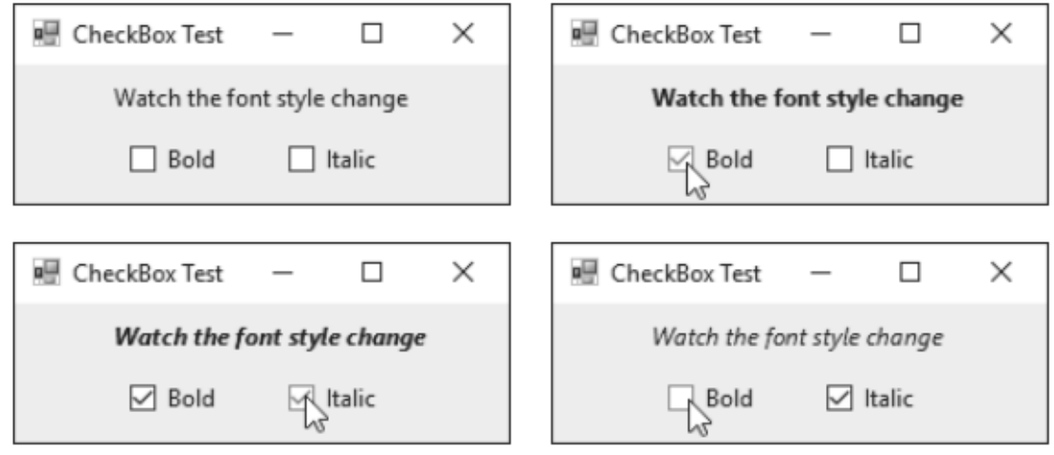
private void redRadioButton_CheckedChanged(object sender, EventArgs e)
{
      if (redRadioButton.Checked)
      {
          this.BackColor = Color.Red;
      }
}

private void whiteRadioButton_CheckedChanged(object sender, EventArgs e)
{
      if (whiteRadioButton.Checked)
      {
          this.BackColor = Color.White;
      }
}

private void normalRadioButton_CheckedChanged(object sender, EventArgs e)
{
      if (normalRadioButton.Checked)
      {
          this.BackColor = SystemColors.Control;
      }
}
```

# Check Boxes

- A check box appears as a small box with some accompanying text.
- They are called check boxes because clicking on an empty check box causes a check mark to appear in it.
- If a check mark already appears in a check box, clicking it removes the check mark.
- Check boxes are similar to radio buttons, except that you can have one or more check boxes in a group, and any number of them can be selected at any given time.



# Check Boxes

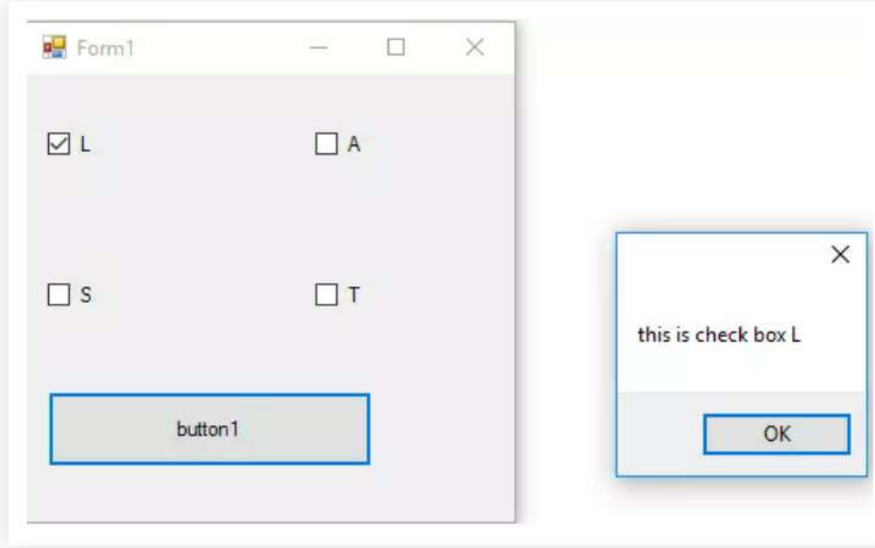
- The CheckBox Control's Text Property: Text property holds the text that is displayed next to the check box.
- The CheckBox Control's Checked Property: when a CheckBox control is selected, or checked, its Checked property is set to True. When a CheckBox control is deselected, or unchecked, its Checked property is set to False.

# Working with CheckBox Controls in Code

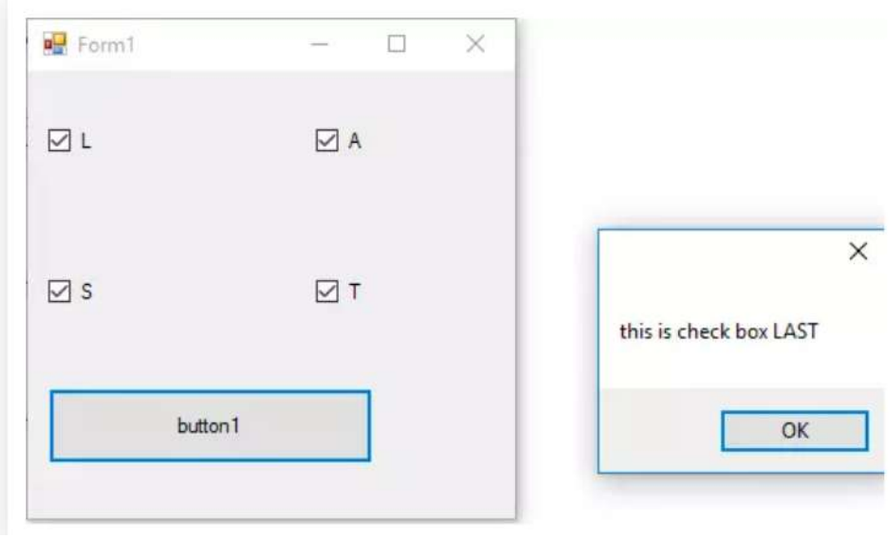
- **Example:** select any of the check boxes and then click the OK button. One or more message boxes will appear, showing you the items that you selected.

```
1 private void okButton_Click(object sender, EventArgs e)
2 {
3     if (pepperoniCheckBox.Checked)
4     {
5         MessageBox.Show("You selected Pepperoni.");
6     }
7
8     if (cheeseCheckBox.Checked)
9     {
10        MessageBox.Show("You selected Cheese.");
11    }
12
13    if (anchoviesCheckBox.Checked)
14    {
15        MessageBox.Show("You selected Anchovies.");
16    }
17 }
```

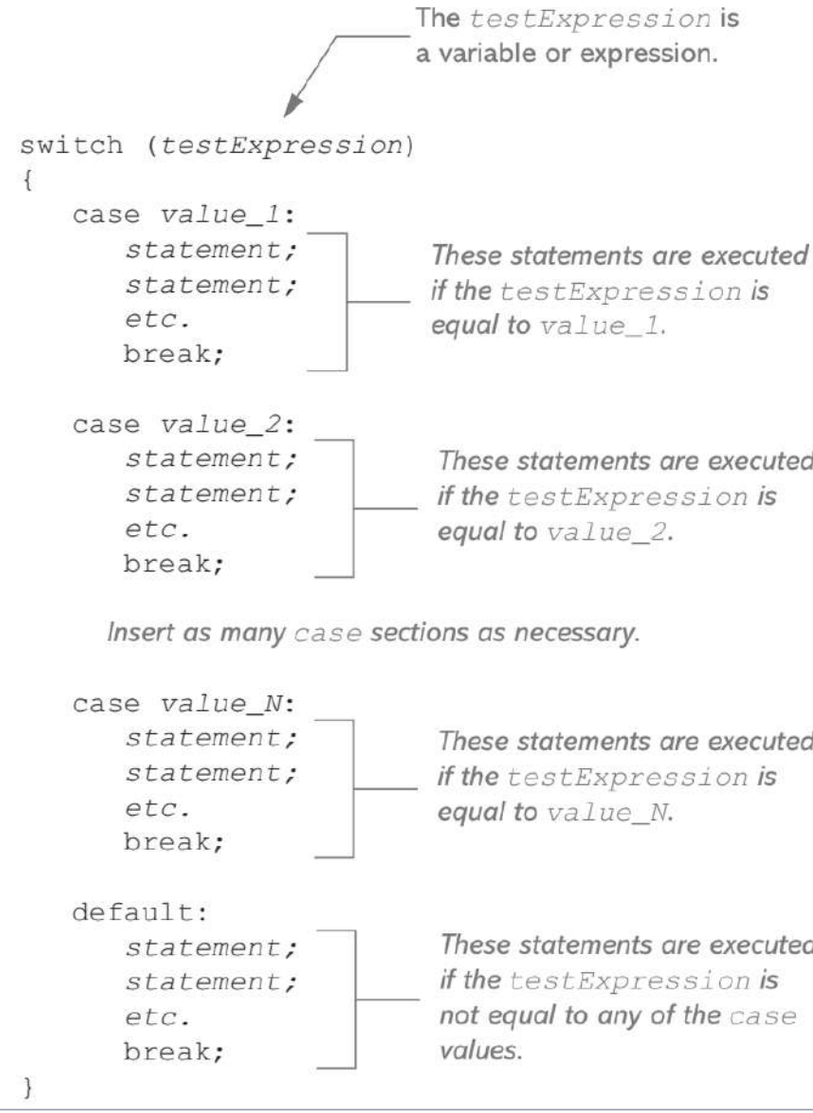
# Problem




When all four CheckBoxes are checked then it will show an output like below.



# Switch Statement



# Switch Statement

- Switch statement works like this if-else-if statement:

```
switch (month)
{
    case 1:
        MessageBox.Show("January");
        break;

    case 2:
        MessageBox.Show("February");
        break;

    case 3:
        MessageBox.Show("March");
        break;

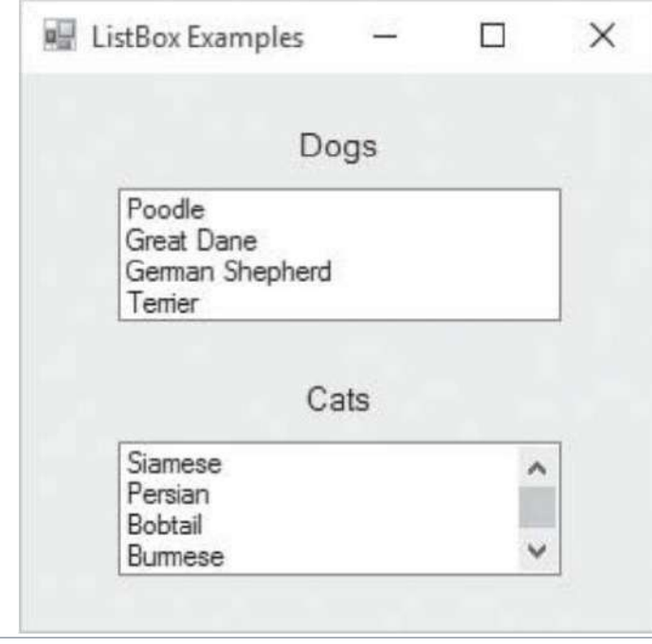
    default:
        MessageBox.Show("Error: Invalid month");
        break;
}
```

```
if (month == 1)
{
    MessageBox.Show("January");
}

else if (month == 2)
{
    MessageBox.Show("February");
}
else if (month == 3)
{
    MessageBox.Show("March");
}
else
{
    MessageBox.Show("Error: Invalid month");
}
```

# List Boxes

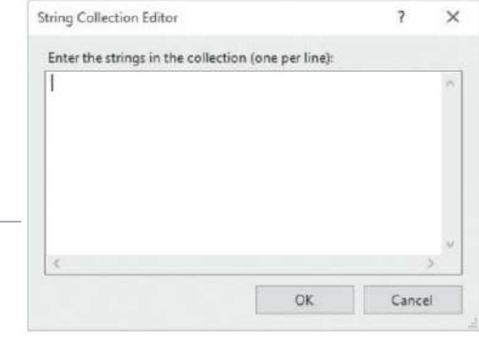
- List boxes display a list of items and allow the user to select an item from the list.
- At run time, the user may select one of the items, causing the item to appear selected.
- The topmost ListBox in the Figure does not have a scroll bar, but the bottom one does.
- A scroll bar appears when a ListBox contains more items than can be displayed in the space provided.



# List Boxes

- Once you create a ListBox control, you add items to its Items property.
- The items that you add to a ListBox's Items property are displayed in the ListBox.
- To store values in the Items property at design time, follow these steps:

1. Select the ListBox control in the Designer window.
2. In the Properties window, the setting for the Items property is displayed as (Collection). When you select the Items property, an ellipses button (...) appears.
3. Click the ellipses button. The String Collection Editor dialog box appears, as shown in the Figure.
4. Type the values that are to appear in the ListBox into the String Collection Editor dialog box. Type each value on a separate line by pressing the Enter key after each entry.
5. When you have entered all the values, click the OK button.



# SelectedItem Property

- When the user selects an item in a ListBox, the item is stored in the ListBox's SelectedItem property.
- For example, suppose an application has a ListBox control named fruitListBox and a string variable named selectedFruit.
- The fruitListBox control contains the items Apples, Pears, and Bananas.
- If the user has selected Pears, the following statement assigns the string "Pears" to the variable selectedFruit:

```
selectedFruit = fruitListBox.SelectedItem.ToString();
```

# Determining Whether an Item Is Selected

- An exception will occur if you try to get the value of a ListBox's SelectedItem property when no item is selected in the ListBox.
- For that reason, you should always make sure that an item is selected before reading the SelectedItem property.
- You do this with the SelectedIndex property.

# SelectedIndex Property

- The items that are stored in a ListBox each have an index.
- The index is simply a number that identifies the item's position in the ListBox.
- The first item has the index 0, the next has the index 1, and so forth.
- The last index value is n − 1, where n is the number of items in the ListBox.
- When the user selects an item in a ListBox, the item's index is stored in the ListBox's SelectedIndex property.
- If no item is selected in the ListBox, the SelectedIndex property is set to −1.

# SelectedIndex Property

- You can use the SelectedIndex property to make sure that an item is selected in a ListBox before you try to get the value of the SelectedItem property.
- **Example:**

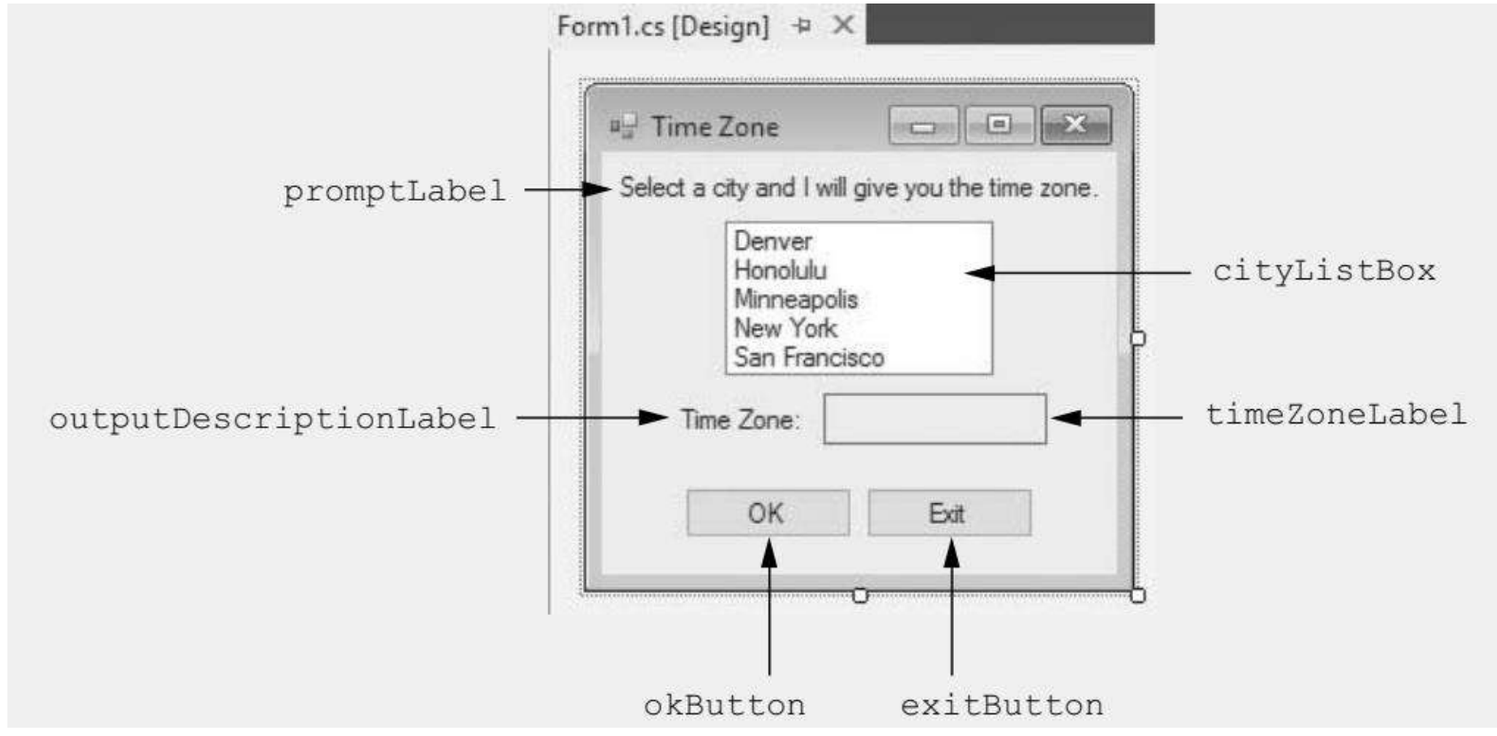
```
if (fruitListBox.SelectedIndex != −1)
{
selectedFruit = fruitListBox.SelectedItem.ToString();
}
```

# Example 9

## ***Time Zone application***

**Create an application that allows the user to select a city from a ListBox control.**

**When the user clicks a button, the application displays the name of the city's time zone.**



# Example 9- Control property settings

| Control Name | Control Type | Property Settings |
|---|---|---|
| `promptLabel` | Label | **Text:** *Select a city and I will give you the time zone.* |
| `cityListBox` | ListBox | **Items:**<br>*Denver*<br>*Honolulu*<br>*Minneapolis*<br>*New York*<br>*San Francisco* |
| `outputDescriptionLabel` | Label | **Text:** *Time Zone:* |
| `timeZoneLabel` | Label | **AutoSize:** False<br>**BorderStyle:** FixedSingle<br>**Text:** (The contents of the Text property have been erased.)<br>**TextAlign:** MiddleCenter |
| `okButton` | Button | **Text:** *OK* |
| `exitButton` | Button | **Text:** *Exit* |

# Example 9- Form1 code for Time Zone application

```
private void okButton_Click(object sender, EventArgs e)
{
    string city;   // To hold the name of a city

    if (cityListBox.SelectedIndex != -1)
    {
        // Get the selected item.
        city = cityListBox.SelectedItem.ToString();
        // Determine the time zone.
        switch (city)
        {
            case "Honolulu":
                timeZoneLabel.Text = "Hawaii-Aleutian";
                break;
            case "San Francisco":
                timeZoneLabel.Text = "Pacific";
                break;
            case "Denver":
                timeZoneLabel.Text = "Mountain";
                break;
            case "Minneapolis":
                timeZoneLabel.Text = "Central";
                break;
            case "New York":
                timeZoneLabel.Text = "Eastern";
                break;
        }
    }
    else
    {
        // No city was selected.
        MessageBox.Show("Select a city.");
    }
}
```

# ListBoxes

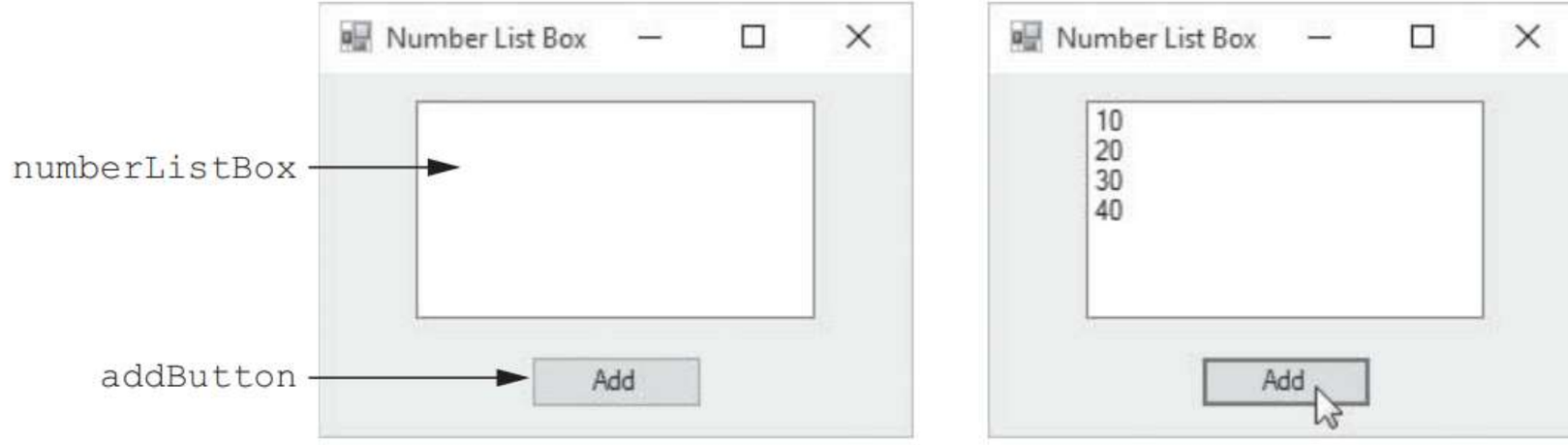
- Previously, we introduced the ListBox control, which displays a list of items and allows the user to select one or more items from the list.
- Previously, you add items to a ListBox control's Items property, and those items are displayed in the ListBox.
- Now, we use ListBox controls to display output.
- At design time, you can use the Properties window to add items to the control's Items property.
- You can also write code that adds items to a ListBox control at run time.
- To add an item to a ListBox control with code, you call the control's Items.Add method.

```
ListBoxName.Items.Add(Item);
```

# Example 10

## ***Number List application***

**When you click the addButton control, the numbers shown in the image on the right are added to the numberListBox control.**



# Example 10- Form1 code for Number List application

```
1 private void addButton_Click(object sender, EventArgs e)
2 {
3       numberListBox.Items.Add(10);
4       numberListBox.Items.Add(20);
5       numberListBox.Items.Add(30);
6       numberListBox.Items.Add(40);
7 }
```

# Items.Count Property

- ListBox controls have an Items.Count property that reports the number of items stored in the ListBox.
- If the ListBox is empty, the Items.Count property equals 0.
- For example, assume an application has a ListBox control named employeesListBox. The following if statement displays a message box if there are no items in the ListBox:

```
if (employeesListBox.Items.Count == 0)
{
MessageBox.Show("There are no items in the list!");
}
```

- The Items.Count property holds an integer value.
- Assuming numEmployees is an int variable, the following statement assigns the number of items in the employeesListBox to the numEmployees variable:

```
numEmployees = employeesListBox.Items.Count;
```

# Items.Clear Method

- ListBox controls have an Items.Clear method that erases all the items in the Items property.

```
ListBoxName.Items.Clear();
```

- For example, assume an application has a ListBox control named employeesListBox.
- The following statement clears all the items in the list:

```
employeesListBox.Items.Clear();
```

# Items.RemoveAt

- The method RemoveAt is used to remove an item from the ListBox

```
ListBox.Items.RemoveAt(Index);
```

# Example 11

***Adds, removes and clears ListBox items Application***

**When the user clicks the Add Button, the new item appears in displayListBox.**

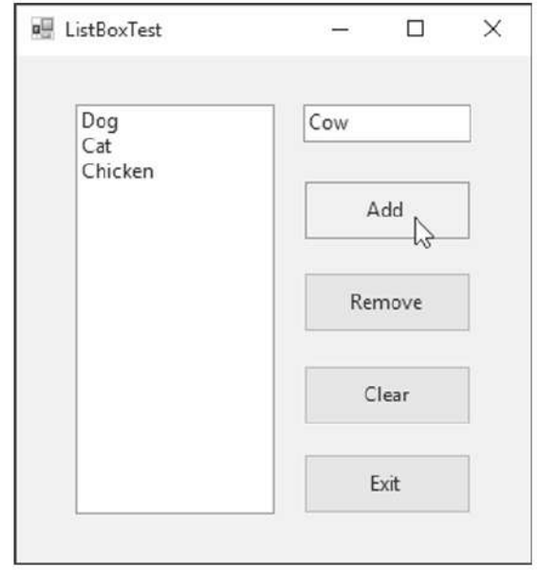
**Similarly, if the user selects an item and clicks Remove, the item is deleted.**

**When clicked, Clear deletes all entries in displayListBox.**

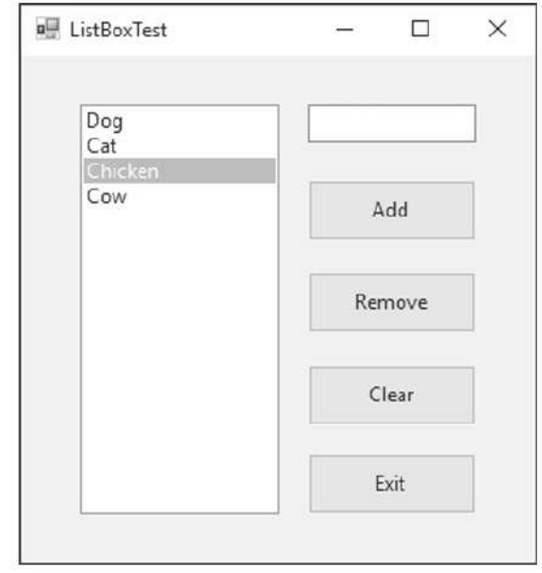
**The user terminates the app by clicking Exit.**

# Example 11

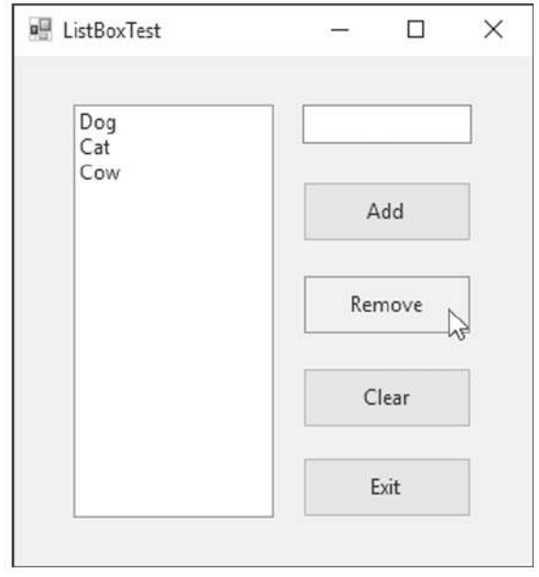
a) GUI after adding **Dog**, **Cat** and **Chicken** and before adding **Cow**



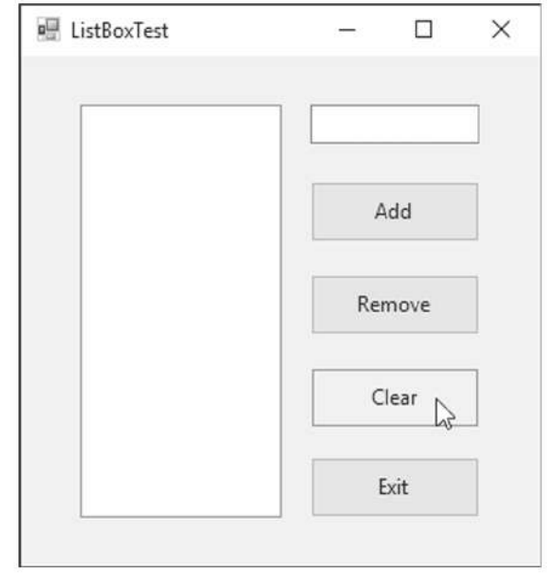

b) GUI after adding **Cow** and before deleting **Chicken**




c) GUI after deleting **Chicken**




d) GUI after clearing the `ListBox`



# Example 11- Form1 code for Adds, removes and clears ListBox items Application

```
// add new item to ListBox (text from input TextBox)
// and clear input TextBox
private void addButton_Click(object sender, EventArgs e)
{
   displayListBox.Items.Add(inputTextBox.Text);
   inputTextBox.Clear();
}

// remove item if one is selected
private void removeButton_Click(object sender, EventArgs e)
{
   // check whether item is selected; if so, remove
   if (displayListBox.SelectedIndex != -1)
   {
      displayListBox.Items.RemoveAt(displayListBox.SelectedIndex);
   }
}

// clear all items in ListBox
private void clearButton_Click(object sender, EventArgs e)
{
   displayListBox.Items.Clear();
}
```


THANK YOU